

RoboRacer Control: MPPI and Nonlinear MPC

Mathematical Overview

Contents

1	Vehicle Model	2
1.1	States and Inputs	2
1.2	Continuous-Time Dynamics	2
1.3	Constraints	2
2	Raceline and Reference Trajectory	2
3	MPPI — Model Predictive Path Integral Control	3
3.1	Conceptual Overview	3
3.2	Discrete-Time Dynamics (Euler)	3
3.3	Algorithm	4
3.4	Cost Function	4
4	Nonlinear MPC	5
4.1	Conceptual Overview	5
4.2	Continuous-Time Dynamics	5
4.3	Optimal Control Problem	5
4.4	Cost Function	6
4.5	SQP Real-Time Iteration (SQP-RTI)	6
5	Throttle Control	7
6	Future Model Improvements	8
6.1	Dynamic Bicycle Model	8
6.2	Pacejka “Magic Formula” Tire Model	8

1 Vehicle Model

Both controllers share the same underlying vehicle model: the **kinematic bicycle model**. It treats the car as a single-track rigid body with no tire slip, making it valid at low-to-moderate speeds where lateral acceleration is small enough that the tires remain near the linear region of their force-slip curves.

1.1 States and Inputs

$$\mathbf{x} = \begin{pmatrix} X \\ Y \\ \psi \\ v \end{pmatrix}, \quad \mathbf{u} = \begin{pmatrix} \delta \\ a \end{pmatrix}$$

Symbol	Description	Unit
X, Y	Global position	m
ψ	Heading angle (yaw)	rad
v	Longitudinal speed	m s ⁻¹
δ	Front-wheel steering angle	rad
a	Longitudinal acceleration	m s ⁻²
L	Wheelbase	m

1.2 Continuous-Time Dynamics

$$\dot{X} = v \cos \psi \tag{1}$$

$$\dot{Y} = v \sin \psi \tag{2}$$

$$\dot{\psi} = \frac{v}{L} \tan \delta \tag{3}$$

$$\dot{v} = a \tag{4}$$

Equation (3) is derived from Ackermann geometry: for a bicycle model the instantaneous turning radius is $R = L / \tan \delta$, so the yaw rate is $\dot{\psi} = v / R$.

1.3 Constraints

$$\delta \in [-\delta_{\max}, \delta_{\max}], \quad a \in [a_{\min}, a_{\max}], \quad v \in [0, v_{\max}]$$

with $\delta_{\max} = 0.44 \text{ rad}$, $a_{\min} = -6.15$, $a_{\max} = 6.15 \text{ m s}^{-2}$, $v_{\max} = 15 \text{ m s}^{-1}$, and $L = 0.324 \text{ m}$.

2 Raceline and Reference Trajectory

A pre-computed raceline CSV provides the reference:

$$\mathcal{R} = \{(X_i, Y_i, \psi_i, v_{\text{ref},i}, s_i)\}_{i=0}^{M-1}$$

where s_i is the cumulative arc length at waypoint i . Given the car’s current position, the **closest waypoint index** i^* is found by exhaustive nearest-neighbour search. Lookahead reference points at horizon step k are obtained by projecting k steps forward in arc length:

$$s_{\text{ref}}(k) = s_{i^*} + k \cdot v_{\text{ref},i^*} \cdot \Delta t$$

and then looking up the corresponding raceline index via binary search on the arc-length array.

3 MPPI — Model Predictive Path Integral Control

3.1 Conceptual Overview

MPPI (Williams et al., 2017) is a **sampling-based** MPC variant rooted in stochastic optimal control and information theory. Instead of solving an optimisation problem, it runs K random *rollouts* of the vehicle model forward in time, scores each one with a cost function, and then computes the optimal control as a cost-weighted average of all the rollout inputs.

On the RoboRacer, at every 25 ms control step:

1. $K = 300$ random perturbation sequences are sampled.
2. Each is propagated through the bicycle model for $N = 20$ steps.
3. Each trajectory is scored: trajectories that stay close to the raceline receive high importance weight; trajectories that deviate are down-weighted.
4. The optimal input is a soft mixture of all sequences, dominated by the best-performing ones.
5. Only the first input of the mixture is applied to the car; the full sequence is shifted and reused as the warm start for the next step.

3.2 Discrete-Time Dynamics (Euler)

MPPI propagates an ensemble of states simultaneously using vectorised forward Euler integration:

$$X_{t+1}^{(k)} = X_t^{(k)} + v_t^{(k)} \cos \psi_t^{(k)} \Delta t \quad (5)$$

$$Y_{t+1}^{(k)} = Y_t^{(k)} + v_t^{(k)} \sin \psi_t^{(k)} \Delta t \quad (6)$$

$$\psi_{t+1}^{(k)} = \psi_t^{(k)} + \frac{v_t^{(k)}}{L} \tan \delta_t^{(k)} \Delta t \quad (7)$$

$$v_{t+1}^{(k)} = \text{clip}\left(v_t^{(k)} + a_t^{(k)} \Delta t, 0, v_{\text{max}}\right) \quad (8)$$

The clip in the velocity update is a practical bound: it is non-differentiable, but MPPI never requires gradients through the dynamics.

3.3 Algorithm

Let $\mathbf{U} = [\mathbf{u}_0, \dots, \mathbf{u}_{N-1}] \in \mathbb{R}^{N \times 2}$ be the current nominal (warm-start) sequence and $\lambda > 0$ the temperature.

Step 1 – Sample perturbations.

$$\boldsymbol{\varepsilon}^{(k)} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}), \quad k = 1, \dots, K$$

with $\boldsymbol{\Sigma} = \text{diag}(\sigma_\delta^2, \sigma_a^2)$, $\sigma_\delta = 0.15 \text{ rad}$, $\sigma_a = 1.5 \text{ m s}^{-2}$.

Step 2 – Form and clip perturbed sequences.

$$\mathbf{V}^{(k)} = \text{clip}\left(\mathbf{U} + \boldsymbol{\varepsilon}^{(k)}, \mathbf{u}_{\min}, \mathbf{u}_{\max}\right)$$

Step 3 – Roll out and accumulate cost. Starting from the current state $\mathbf{x}_0$, propagate each sequence and accumulate the running cost:

$$J^{(k)} = \sum_{t=0}^{N-1} c\left(\mathbf{x}_t^{(k)}, \mathbf{V}_t^{(k)}\right)$$

Step 4 – Compute importance weights. Subtract the minimum for numerical stability, then normalise:

$$\beta = \min_k J^{(k)}, \quad \tilde{w}_k = \exp\left(-\frac{J^{(k)} - \beta}{\lambda}\right), \quad w_k = \frac{\tilde{w}_k}{\sum_j \tilde{w}_j}$$

Low temperature λ concentrates weight on the best rollouts (greedy); high temperature spreads it more evenly (exploratory).

Step 5 – Update nominal sequence.

$$\mathbf{U}^* = \sum_{k=1}^K w_k \mathbf{V}^{(k)}$$

Step 6 – Apply and shift. Apply $\mathbf{u}_0^*$ to the car. Shift the sequence left by one step and zero-pad the tail to form the warm start for the next control iteration.

3.4 Cost Function

$$c(\mathbf{x}_t, \mathbf{u}_t) = W_{\text{CTE}} \underbrace{\|\mathbf{p}_t - \mathbf{p}_{\text{ref}}(t)\|_2}_{\text{cross-track error}} + W_h \underbrace{|\psi_t - \psi_{\text{ref}}(t)|_\pi}_{\text{heading error}} + W_v \underbrace{|v_t - v_{\text{ref}}(t)|}_{\text{speed error}} + W_s \delta_t^2$$

where $|\cdot|_\pi$ denotes the wrapped absolute angle difference and $\mathbf{p}_{\text{ref}}(t)$, $\psi_{\text{ref}}(t)$, $v_{\text{ref}}(t)$ are looked up from the raceline at the projected arc position.

Weight	Value	Meaning
W_{CTE}	5.0	Penalise lateral deviation from the raceline
W_h	2.0	Penalise heading misalignment
W_v	1.0	Penalise speed error relative to v_{ref}
W_s	5.0	Penalise harsh steering inputs

4 Nonlinear MPC

4.1 Conceptual Overview

Nonlinear MPC frames vehicle control as a **constrained optimisation problem** solved at every timestep. Rather than sampling random trajectories, it *solves* for the single best input sequence that minimises a quadratic cost subject to the vehicle dynamics and hard input/state constraints.

On the RoboRacer, at every 25 ms control step:

1. The current Vicon pose is fed to the acados solver as the initial condition.
2. The reference trajectory for the next N prediction steps is looked up from the raceline.
3. The solver performs one SQP-RTI step: it linearises the nonlinear dynamics around the warm-started trajectory, solves the resulting quadratic programme (QP) using HPIPM, and returns the optimal input.
4. Only the first input of the sequence is applied; the rest warm-start the next solve.

4.2 Continuous-Time Dynamics

Unlike MPPI, the MPC uses the continuous-time ODE (1)–(4) directly. acados integrates it numerically using explicit Runge–Kutta 4 (ERK4):

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\Delta t}{6} (\mathbf{f}_1 + 2\mathbf{f}_2 + 2\mathbf{f}_3 + \mathbf{f}_4)$$

where $\mathbf{f}_1, \dots, \mathbf{f}_4$ are the standard RK4 stage derivatives of $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$. This is more accurate than forward Euler and does not require the velocity clip; speed bounds are instead expressed as explicit inequality constraints:

$$0 \leq v_k \leq v_{\text{max}} \quad \forall k = 1, \dots, N$$

4.3 Optimal Control Problem

At time t with current state $\mathbf{x}_0$:

$$\begin{aligned}
& \min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + \ell_N(\mathbf{x}_N) & (9) \\
& \text{subject to } \mathbf{x}_{k+1} = F(\mathbf{x}_k, \mathbf{u}_k) & \text{(dynamics)} \\
& \mathbf{u}_k \in [\mathbf{u}_{\min}, \mathbf{u}_{\max}] & \text{(input bounds)} \\
& v_k \in [0, v_{\max}] & \text{(speed bound)} \\
& \mathbf{x}_0 = \mathbf{x}_{\text{current}} & \text{(initial condition)}
\end{aligned}$$

where $F(\mathbf{x}_k, \mathbf{u}_k)$ denotes one ERK4 step of the bicycle ODE.

4.4 Cost Function

A Linear Least-Squares (LINEAR_LS) cost is used:

$$\ell(\mathbf{x}_k, \mathbf{u}_k) = \|\mathbf{y}_k - \mathbf{y}_{\text{ref},k}\|_{\mathbf{W}}^2 = (\mathbf{y}_k - \mathbf{y}_{\text{ref},k})^\top \mathbf{W} (\mathbf{y}_k - \mathbf{y}_{\text{ref},k})$$

with the residual vector

$$\mathbf{y}_k = \begin{pmatrix} X_k \\ Y_k \\ \psi_k \\ v_k \\ \delta_k \\ a_k \end{pmatrix}, \quad \mathbf{y}_{\text{ref},k} = \begin{pmatrix} X_{\text{ref},k} \\ Y_{\text{ref},k} \\ \psi_{\text{ref},k} \\ v_{\text{ref},k} \\ 0 \\ 0 \end{pmatrix}, \quad \mathbf{W} = \text{diag}(W_{\text{CTE}}, W_{\text{CTE}}, W_h, W_v, W_s, W_a)$$

The zero reference for δ and a means the solver penalises control effort (regularisation), keeping the trajectory smooth. The terminal cost uses the same structure with $\mathbf{W}_N = \text{diag}(W_{\text{CTE}}, W_{\text{CTE}}, W_h, W_v)$.

Weight	Value	Meaning
W_{CTE}	5.0	Position tracking (applied to both X and Y residuals)
W_h	2.0	Heading tracking
W_v	1.0	Speed tracking
W_s	5.0	Steering regularisation
W_a	0.1	Acceleration regularisation

4.5 SQP Real-Time Iteration (SQP-RTI)

Solving the full NLP (9) to convergence at 40 Hz is infeasible for large horizons. The **Real-Time Iteration** (Diehl et al., 2005) scheme replaces the full NLP solve with a single SQP step per control cycle.

At each step, the NLP is linearised around the current warm-start trajectory $(\bar{\mathbf{X}}, \bar{\mathbf{U}})$ to obtain a **quadratic programme**:

$$\begin{aligned}
& \min_{\Delta \mathbf{U}} \quad \frac{1}{2} \Delta \mathbf{U}^\top \mathbf{H} \Delta \mathbf{U} + \mathbf{g}^\top \Delta \mathbf{U} & (10) \\
& \text{subject to} \quad \mathbf{C} \Delta \mathbf{U} = \mathbf{c} & \text{(linearised dynamics)} \\
& \quad \mathbf{u}_{\min} - \bar{\mathbf{U}} \leq \Delta \mathbf{U} \leq \mathbf{u}_{\max} - \bar{\mathbf{U}} & \text{(input bounds)}
\end{aligned}$$

where $\mathbf{H}$ is the Gauss–Newton Hessian approximation and $\mathbf{g}$ the gradient of the cost. The QP is solved by **HPIPM** (partial condensing), and the solution $\Delta \mathbf{U}^*$ is added to the warm start.

Because the initial state changes only slightly between control steps (the car has moved 25 ms worth), the warm-started trajectory is already close to the new optimum and one SQP step is sufficient to track it accurately.

5 Throttle Control

Neither controller directly commands a wheel torque. Both use a **feedforward + proportional speed controller** that maps the raceline reference velocity to a dimensionless throttle integer (0–200):

$$\tau = \text{clip}(K_{\text{ff}} v_{\text{ref}} + K_p (v_{\text{ref}} - \hat{v}), 0, \tau_{\max})$$

where $\hat{v}$ is the speed estimate from Vicon position differences (EMA-smoothed), $K_{\text{ff}} = 20$ and $K_p = 5$ are default gains. The MPPI acceleration output a and the MPC acceleration command a^* are not used for throttle for four compounding reasons:

1. **No calibrated actuator model.** Both controllers output a in m s^{-2} , but the hardware input is a dimensionless throttle integer sent to an ESC. The mapping throttle $\rightarrow$ torque $\rightarrow$ acceleration varies with speed, battery voltage, and surface friction. Without empirical system identification a direct $a \rightarrow \tau$ conversion is unreliable.
2. **Noisy velocity estimate.** $\hat{v}$ is computed from finite differences of Vicon positions and is EMA-smoothed but still noisy. That noise enters the MPPI/MPC initial state, propagates through all rollouts or linearisations, and produces a noisy a command. A simple proportional controller on $v_{\text{ref}} - \hat{v}$ is far more forgiving of this noise than closing the throttle loop through the optimiser.
3. **Speed profile already solved offline.** The raceline CSV encodes a pre-optimised speed profile (slow for tight corners, fast on straights). Re-optimising speed online adds complexity without benefit when the reference profile is already well-designed; the controller need only track it.
4. **Decoupled tuning.** At low slip angles, lateral and longitudinal dynamics interact weakly. Keeping the speed loop separate means K_{ff} and K_p can be tuned independently of the MPPI hyper-parameters or MPC weights, making both easier to diagnose.

The steering servo command (integer 120–880) is derived from the controller’s steering angle output in both cases:

$$\tau_\delta = \text{clip}(\tau_{\text{center}} + K_{\text{srv}} \delta + \tau_{\text{trim}}, \tau_{\min}, \tau_{\max})$$

with $\tau_{\text{center}} = 512$, $K_{\text{srv}} = 1200$ servo-units/rad, $\tau_{\text{trim}} = 20$.

6 Future Model Improvements

6.1 Dynamic Bicycle Model

The kinematic model assumes zero tire slip, which is only valid when lateral acceleration $a_y \ll \mu g$. At higher speeds the tires develop meaningful slip angles and the kinematic model underestimates the required steering.

The **dynamic bicycle model** adds lateral velocity v_y and yaw rate $r = \dot{\psi}$ as states, and replaces the kinematic steering constraint with force balance equations:

$$\mathbf{x} = \begin{pmatrix} X \\ Y \\ \psi \\ v_x \\ v_y \\ r \end{pmatrix}, \quad m\dot{v}_y = F_{y,f} + F_{y,r} - mv_x r, \quad I_z \dot{r} = L_f F_{y,f} - L_r F_{y,r}$$

where m is vehicle mass, I_z yaw moment of inertia, L_f/L_r are distances from CG to front/rear axles, and $F_{y,f}$, $F_{y,r}$ are lateral tire forces. For a linear tire model: $F_{y,f} = -C_f \alpha_f$, $F_{y,r} = -C_r \alpha_r$, with slip angles

$$\alpha_f = \delta - \arctan\left(\frac{v_y + L_f r}{v_x}\right), \quad \alpha_r = -\arctan\left(\frac{v_y - L_r r}{v_x}\right)$$

and cornering stiffnesses C_f , C_r identified from vehicle tests.

6.2 Pacejka “Magic Formula” Tire Model

For cornering beyond the linear regime (slipping or near-limit driving), the linear tire force model fails. The **Pacejka magic formula** (Pacejka, 2012) captures the full nonlinear force–slip curve:

$$F_y(\alpha) = D \sin\left[C \arctan\left(B\alpha - E(B\alpha - \arctan(B\alpha))\right)\right]$$

where B (stiffness factor), C (shape factor), D (peak value), and E (curvature factor) are tire-specific parameters. At small slip angles the formula reduces to the linear model ($F_y \approx BCD\alpha$, so $C_{\text{eff}} = BCD$), ensuring consistency with the linear model.

Replacing the linear tire forces in the dynamic bicycle model with the Pacejka formula gives a **quarter-car + Pacejka** model that is accurate from low-speed cruising through near-limit cornering. Both MPPI and the MPC formulation can accommodate it without structural changes: MPPI requires only that the forward model be evaluable; MPC requires additionally that F_y is differentiable, which the smooth Pacejka formula satisfies.